AI Assisted Coding

ASSIGNMENT : 12.2

1

Name : L. Sharan Sai Varshith

bt.no : 21

Ht.No : 2303A51450

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

➢ Task: Use AI assistance to generate a Python program that

implements a Stack data structure. Instructions:

Prompt AI to create a Stack class with the following methods:

➢ push(element) ➢ pop()

➢ peek() ➢ is_empty()

➢ Ensure proper error handling for stack underflow.

➢ Ask AI to include clear docstrings

for each method. Expected Output:

➢ A functional Python program implementing a Stack using a class.

➢ Properly documented methods

with docstrings.

Code:

```python
from collections import deque
class Stack:
    """
    A class to represent a Stack data structure using a deque for efficient operations.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = deque()
    def push(self, element):
        """
        Add an element to the top of the stack.
        Parameters:
        element: The element to be added to the stack.
        """
        self.stack.append(element)
    def pop(self):
        """
        Remove and return the top element of the stack.
        Returns:
        The top element of the stack if it is not empty; otherwise, raises an exception.
        Raises:
        IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
        return self.stack.pop()
    def peek(self):
        """
        Return the top element of the stack without removing it.
        Returns:
        The top element of the stack if it is not empty; otherwise, raises an exception.
        Raises:
        IndexError: If the stack is empty (stack underflow).
        """
        if self.is_empty():
            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
        return self.stack[-1]
```

```python
    def is_empty(self):
        """
        Check if the stack is empty.
        Returns:
        True if the stack is empty; otherwise, False.
        """
        return len(self.stack) == 0
# Example usage of the Stack class
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Top element (peek):", stack.peek())  # Output: 3
    print("Popped element:", stack.pop())        # Output: 3
    print("Is stack empty?", stack.is_empty())   # Output: False
    print("Popped element:", stack.pop())        # Output: 2
    print("Popped element:", stack.pop())        # Output: 1
    print("Is stack empty?", stack.is_empty())   # Output: True
    try:
        stack.pop()  # This will raise an exception since the stack is empty
    except IndexError as e:
        print(e)  # Output: Stack underflow: Cannot pop from an empty stack.
    try:
        stack.peek()  # This will also raise an exception since the stack is empty
    except IndexError as e:
        print(e)  # Output: Stack underflow: Cannot peek from an empty stack.
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Top element (peek): 3
Popped element: 3
Is stack empty? False
Popped element: 2
Popped element: 1
Is stack empty? True
Stack underflow: Cannot pop from an empty stack.
Stack underflow: Cannot peek from an empty stack.
```

Observation:

Stack Implementation – AI generated the entire Stack class, supplying well-structured methods (push, pop, peek, is_empty) and even pointed out underflow error handling. Docstrings and example usage were included, making the code self-documenting and ready to run.

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

➢ Task: Use AI to implement and compare Linear Search and Binary

Search algorithms in Python. Instructions:

➢ Prompt AI to generate:

➢ linear_search(arr, target)

➢ binary_search(arr, target)

➢ Include docstrings explaining:

➢ Working principle

➢ Test both algorithms using

different input sizes.

➢ Expected Output:

- Python implementations of both search algorithms.
- AI-generated comments and complexity analysis.
- Test results showing correctness and comparison.

Code :

```python
def linear_search(arr, target):
    """Parameters:
    arr (list): The list in which to search for the target element.
    target: The element to search for.
    Returns:
    int: The index of the target element if found; otherwise, -1.
    Complexity:
    Time Complexity: O(n) - In the worst case, we may have to check each element once.
    Space Complexity: O(1) - No additional space is used.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
def binary_search(arr, target):
    """ Parameters:
    arr (list): The sorted list in which to search for the target element.
    target: The element to search for.
    Returns:
    int: The index of the target element if found; otherwise, -1.
    Complexity:
    Time Complexity: O(log n) - The search space is halved with each iteration.
    Space Complexity: O(1) - No additional space is used.
    Note:
    The input array must be sorted for binary search to work correctly.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

```python
# Test both algorithms with different input sizes
if __name__ == "__main__":
    # Test data
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    target = 5

    # Testing Linear Search
    print("Testing Linear Search:")
    index_linear = linear_search(arr, target)
    if index_linear != -1:
        print(f"Element {target} found at index {index_linear} using Linear Search.")
    else:
        print(f"Element {target} not found using Linear Search.")

    # Testing Binary Search
    print("\nTesting Binary Search:")
    index_binary = binary_search(arr, target)
    if index_binary != -1:
        print(f"Element {target} found at index {index_binary} using Binary Search.")
    else:
        print(f"Element {target} not found using Binary Search.")
    # Testing with a larger input size
    large_arr = list(range(1, 10001))  # A sorted array from 1 to 10000
    target_large = 9999
    print("\nTesting with a larger input size:")
    index_linear_large = linear_search(large_arr, target_large)
    index_binary_large = binary_search(large_arr, target_large)
    print(f"Element {target_large} found at index {index_linear_large} using Linear Search.")
    print(f"Element {target_large} found at index {index_binary_large} using Binary Search.")
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Testing Linear Search:
Element 5 found at index 4 using Linear Search.

Testing Binary Search:
Element 5 found at index 4 using Binary Search.

Testing with a larger input size:
Element 9999 found at index 9998 using Linear Search.
Element 9999 found at index 9998 using Binary Search.
```

Observation :

Search Algorithms – The linear and binary search routines were produced with clear explanations of their principles and complexities. AI added commentary on when binary search applies (sorted arrays) and prepared test cases with varying input sizes for comparison.

Task Description -3 (Test Driven Development – Simple Calculator Function)

➢ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.
Instructions:

➢ Prompt AI to first generate unit test cases for addition and subtraction.

➢ Run the tests and observe failures.

➢ Ask AI to implement the calculator functions to pass all tests.

➢ Re-run the tests to

confirm success. Expected

Output:

➢ Separate test file and implementation file. ➢ Test cases executed before implementation.

➢ Final implementation passing

all test cases.

Code:

```python
# test_calculator.py
import unittest
from calculator import add, subtract
class TestCalculator(unittest.TestCase):
    def test_add(self):
        """Test the add function with various cases."""
        self.assertEqual(add(1, 2), 3)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(0, 0), 0)
        self.assertEqual(add(-5, -5), -10)
    def test_subtract(self):
        """Test the subtract function with various cases."""
        self.assertEqual(subtract(5, 2), 3)
        self.assertEqual(subtract(1, -1), 2)
        self.assertEqual(subtract(0, 0), 0)
        self.assertEqual(subtract(-5, -5), 0)
if __name__ == '__main__':
    unittest.main()
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
..
----------------------------------------------------------------
Ran 2 tests in 0.001s

OK
```

Observation:

Test-Driven Calculator – AI first wrote unit tests for add and subtract functions, prompting the classic TDD cycle: write failing tests, then implement the functions to satisfy them. The scaffolding for a separate test_calculator.py was created, demonstrating how AI can jump-start a TDD workflow.

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

Use AI assistance to generate a Python program that implements a

Queue data structure. Instructions:

➢ Prompt AI to create a Queue class with the following methods:

- enqueue(element)
- dequeue()
- front()
- is_empty()

➢ Handle queue overflow and underflow conditions.

➢ Include appropriate docstrings

for all methods. Expected Output:

➢ A fully functional Queue

implementation in Python.

Code:

```python
from collections import deque
class Queue:
    """
    A class to represent a Queue data structure using a deque for efficient operations.
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.queue = deque()
    def enqueue(self, element):
        """
        Add an element to the end of the queue.
        Parameters:
        element: The element to be added to the queue.
        """
        self.queue.append(element)
    def dequeue(self):
        """
        Remove and return the front element of the queue.
        Returns:
        The front element of the queue if it is not empty; otherwise, raises an exception.
        Raises:
        IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot dequeue from an empty queue.")
        return self.queue.popleft()
    def front(self):
        """
        Return the front element of the queue without removing it.
        Returns:
        The front element of the queue if it is not empty; otherwise, raises an exception.
        Raises:
        IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot access front of an empty queue.")
        return self.queue[0]
```

```python
    def is_empty(self):
        """
        Check if the queue is empty.
        Returns:
        True if the queue is empty; otherwise, False.
        """
        return len(self.queue) == 0
# Example usage of the Queue class
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Front element:", queue.front())  # Output: 1
    print("Dequeued element:", queue.dequeue())  # Output: 1
    print("Is queue empty?", queue.is_empty())  # Output: False
    print("Dequeued element:", queue.dequeue())  # Output: 2
    print("Dequeued element:", queue.dequeue())  # Output: 3
    print("Is queue empty?", queue.is_empty())  # Output: True
    try:
        queue.dequeue()  # This will raise an exception since the queue is empty
    except IndexError as e:
        print(e)  # Output: Queue underflow: Cannot dequeue from an empty queue.
    try:
        queue.front()  # This will also raise an exception since the queue is empty
    except IndexError as e:
        print(e)  # Output: Queue underflow: Cannot access front of an empty queue.
```

Output:

```
C:\Users\acera\Desktop\Btech 3 2\AI Assistant coding>python
Front element: 1
Dequeued element: 1
Is queue empty? False
Dequeued element: 2
Dequeued element: 3
Is queue empty? True
Queue underflow: Cannot dequeue from an empty queue.
Queue underflow: Cannot access front of an empty queue.
```

Observation:
Queue Implementation – AI crafted a full Queue class with enqueue, dequeue, front, and is_empty methods, mirroring the earlier stack example. Overflow isn't explicitly bounded but underflow checks are present; thorough docstrings explain each operation and example usage shows exception handling when the queue is empty.

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)

➢ Task:

Use AI to implement Bubble Sort and

Selection Sort algorithms and compare their

behavior.

Instructions:

➢ Prompt AI to generate:

• bubble_sort(arr) • selection_sort(arr)

➢ Include comments explaining each step.

➢ Add docstrings mentioning time and

space complexity. Expected Output:

• Correct Python implementations of both sorting algorithms.

• Complexity analysis in

docstrings.

Code:

```python
def bubble_sort(arr):
    """
    Sort an array using the Bubble Sort algorithm.
    Parameters:
    arr (list): The list to be sorted.
    Returns:
    list: The sorted list.
    Complexity:
    Time Complexity: O(n^2) - In the worst case, we need to compare each element with every other element.
    Space Complexity: O(1) - No additional space is used for sorting (in-place).
    """
    n = len(arr)
    for i in range(n):
        # Flag to check if any swapping occurs
        swapped = False
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break
    return arr
```

```python
def selection_sort(arr):
    """
    Sort an array using the Selection Sort algorithm.
    Parameters:
    arr (list): The list to be sorted.
    Returns:
    list: The sorted list.
    Complexity:
    Time Complexity: O(n^2) - In the worst case, we need to find the minimum element for each position.
    Space Complexity: O(1) - No additional space is used for sorting (in-place).
    """
    n = len(arr)
    for i in range(n):
        # Assume the minimum is the first element of the unsorted part
        min_index = i
        for j in range(i+1, n):
            # Update min_index if the current element is smaller than the minimum
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the found minimum element with the first element of the unsorted part
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
# Example usage of the sorting algorithms
if __name__ == "__main__":
    arr1 = [64, 34, 25, 12, 22, 11, 90]
    arr2 = arr1.copy()

    print("Original array:", arr1)

    sorted_arr_bubble = bubble_sort(arr1)
    print("Sorted array using Bubble Sort:", sorted_arr_bubble)

    sorted_arr_selection = selection_sort(arr2)
    print("Sorted array using Selection Sort:", sorted_arr_selection)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python -u "
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
Sorted array using Selection Sort: [11, 12, 22, 25, 34, 64, 90]
```

Observation:

Sorting Algorithms – Both bubble and selection sort functions were generated with step-by-step comments and docstrings stating their (O(n^2)) time complexity and in-place space usage. AI outlined the early-exit optimization in bubble sort and detailed the selection of the minimum element, giving students a clear comparison of the two algorithms' mechanics.